



CURSO DE FIRMWARE

C para Sistemas Embebidos de 32 bits

"Fundamentos Sólidos para Firmware Seguro y Eficiente"

José Tepetla
Ing. de Software Senior



2026

Objetivos del Curso

✗ Problemas Comunes

- ✗ Variables que "cambian de tamaño" al compilar en otro micro
- ✗ Bucles optimizados que el compilador "elimina"
- ✗ Estructuras que desperdician memoria por padding
- ✗ Punteros que generan Hard Faults
- ✗ Código no determinista que falla en campo

✓ Lo que Logharemos

- ✓ Tipos de datos portables (`stdint.h`)
- ✓ Manejo seguro de variables con `volatile`
- ✓ Estructuras optimizadas sin padding
- ✓ Punteros seguros y encapsulamiento con `static`
- ✓ Uniones para mapeo de protocolos
- ✓ Código UL 60730 compliant

PROYECTO FINAL: Módulo de Control de Seguridad

8 módulos → 8 retos prácticos → 1 proyecto integrador



Temario - 8 Módulos

MÓDULO 1

Fundamentos y Tipos de Datos

stdint.h, overflow, operadores

● RETO: Overflow silencioso

MÓDULO 2

Lógica Condicional

if/else, cortocircuito, bits vs lógicos

● RETO: Interlock de seguridad

MÓDULO 3

Bucles y Switch

for, while, do-while, switch-case

● RETO: Máquina de estados

MÓDULO 4

Funciones y Enumeraciones

Prototipos, paso por valor, enum

● RETO: Configurador de motor

MÓDULO 5

Punteros y Referencias

&, *, paso por referencia, NULL

● RETO: Modificador de contador

MÓDULO 6

static y volatile

Almacenamiento crítico, ISR

● RETO: Simulación de interrupción

MÓDULO 7

struct, union, sizeof

Agrupación, mapeo, tamaño

● RETO: Mapeador de protocolos

MÓDULO 8

Padding, Macros, Inline

Alineación, #define, inline

● RETO: Optimización de estructura

Etapas de desarrollo del software

Por lo general, el desarrollo de software abarca las siguientes etapas:

- **Análisis**
- **Diseño**
- Programación
- **Codificación**
- Prueba
- Mantenimiento
- **Documentación**

El análisis y diseño dependen del tamaño y la finalidad del programa; la programación es escribir la solución propuesta en el idioma nativo, la codificación consiste en escribir el programa con instrucciones en un lenguaje de programación, en este caso en C, considerando el diseño (módulos, algoritmos, diagramas). En ocasiones se emplea indistintamente codificación y programación para designar esta fase del proceso. Posteriormente se compila y prueba el código; se resuelven los errores de sintaxis, en caso de existir, y de lo contrario se procede a la ejecución del programa para probarlo y verificar que se obtenga lo planeado. Se detectan y corrigen los posibles errores de lógica. Una vez terminado el programa, puede requerir mantenimiento para realizar cambios o ajustes relativamente pequeños.

Estructura de un Programa en C

Estructura Estándar

```
// 1. Directivas de preprocesador
#include <stdint.h>

// 2. Definiciones de constantes
#define MAX_TEMP 100

// 2.1 Variables constantes
const uint32_t contador = 10;

// 3. Variables globales
uint32_t contador_global = 0;

// 4. Prototipos de funciones
void inicializar(void);

// 5. Función principal
int main(void) {
    inicializar();
    return 0;
}

// 6. Definición de funciones
void inicializar(void) {
    // Código
}
```

Ámbito y Ciclo de Vida

Tipo	Ámbito	Ciclo de Vida
Definición	Todo el archivo	Todo el programa: Se reemplaza al compilar
Constante	Todo el archivo	Todo el programa: Ocupa espacio en memoria
Local	Bloque { }	Entra/sale del bloque
Global	Todo el archivo	Todo el programa
static local	Función	Todo el programa
static global	Archivo .c	Todo el programa

💡 **Regla:** Minimizar globales. Usar `static` para encapsular.

⚠ Tipos Nativos vs `stdint.h`

❌ Tipos Nativos (Peligrosos)

Tipo	ARM 32b	PIC 16b
<code>int</code>	32 bits	16 bits
<code>short</code>	16 bits	16 bits
<code>long</code>	32 bits	32 bits
<code>char</code>	8 bits	8 bits

⚠ ¡El tamaño **NO** es fijo!

✅ Tipos Explícitos (`stdint.h`)

Tipo	Tamaño	Rango
<code>void</code>	Null o vacío	-
<code>int8_t</code>	8 bits	-128 a 127
<code>uint8_t</code>	8 bits	0 a 255
<code>int16_t</code>	16 bits	-32,768 a 32,767
<code>uint32_t</code>	32 bits	0 a 4,294,967,295
<code>int32_t</code>	32 bits	-2,147,483,648 a 2,147,483,647

✅ **SIEMPRE** usar estos tipos en embebido

Palabras reservadas

Cabe destacar que hay nombres que no se pueden utilizar por ser palabras reservadas del lenguaje C.

Estos nombres prohibidos se presentan en la siguiente tabla.

- | | | |
|------------|------------|------------|
| • auto | • do | • goto |
| • signed | • unsigned | • break |
| • double | • if | • sizeof |
| • void | • case | • else |
| • int | • static | • volatile |
| • char | • enum | • long |
| • struct | • while | • const |
| • extern | • register | • switch |
| • continue | • float | • return |
| • typedef | • default | • for |
| • short | • union | |



Operadores Aritméticos y de Asignación

Los operadores aritméticos nos permiten realizar operaciones entre operandos: números, constantes o variables. El resultado de una operación aritmética siempre es un número.

+ Aritméticos

- + Suma
- - Resta
- * Multiplicación
- / División
- % Módulo (residuo)

✎ Asignación

- = Asignación simple
- += Suma y asigna
- ++ Incrementa 1 unidad
- -= Resta y asigna
- -- Decrementa 1 unidad
- *= Multiplica y asigna
- /= Divide y asigna

💡 Ejemplo

```
uint16_t temperatura = 25;  
temperatura += 5; // 30  
temperatura *= 2; // 60
```

Debemos respetar la jerarquía de los operadores y aplicarlos de izquierda a derecha. Si una expresión contiene subexpresiones entre paréntesis, éstas se evalúan primero.

⚠ **OJO con el overflow:** `uint8_t` $255 + 1 = 0$ (desbordamiento)

Operadores Relacionales

Los operadores relacionales se utilizan para comparar dos operandos, que pueden ser números, caracteres, cadenas de caracteres, constantes o variables.

Operador relacional	Operación	Ejemplos	Resultados
<code>==</code>	Igual a	<code>h == p</code>	<code>false</code>
<code>!=</code>	Diferente de	<code>a != b</code>	<code>true</code>
<code><</code>	Menor que	<code>5 < 10</code>	<code>true</code>
<code>></code>	Mayor que	<code>22 > 11</code>	<code>true</code>
<code><=</code>	Menor o igual que	<code>15 <= 2</code>	<code>false</code>
<code>>=</code>	Mayor o igual que	<code>35 >= 11</code>	<code>true</code>

12 34 Enumeraciones - Parámetros Legibles

Otra forma de nombrar constantes es utilizando el método enumerador: `enum`. Los valores en este caso se asignan de manera predeterminada en incrementos unitarios, comenzando con el cero. Es útil entonces cuando queremos definir constantes con valores predeterminados.

✓ Usar `enum`

```
typedef enum {
    VEL_BAJA = 0,
    VEL_MEDIA = 1,
    VEL_ALTA = 2,
    VEL_MAX = 3
} VelocidadMotor_t;

bool motor_set_velocidad(VelocidadMotor_t velocidad) {
    if (velocidad >= VEL_MAX) {
        return false; // Error
    }
    // Configurar hardware...
    return true;
}
```

✓ Auto-documentado, evita números mágicos

✗ Números Mágicos

```
// ✗ ¿Qué significa 0, 1, 2?
void configurar_motor(int velocidad) {
    if (velocidad == 0) { // ¿Baja?
        // ...
    } else if (velocidad == 1) { // ¿Media?
        // ...
    }
}
```

✗ Difícil de leer, propenso a errores

RETO: El Desbordamiento Silencioso

 **OBJETIVO:** Demostrar overflow y corregir con el tipo correcto

ENUNCIADO:

"Crea un programa que sume dos variables `uint8_t` que provoquen un desbordamiento (overflow) por superar 255. Analiza el resultado y corrígelo eligiendo el tipo de dato explícito correcto."

```
// 📄 CÓDIGO BASE
#include <stdint.h>
#include <stdio.h>

int main() {
    // COMPLETAR: Declarar dos variables uint8_t
    // COMPLETAR: Asignar valores que sumen > 255
    // COMPLETAR: Mostrar resultado
    // COMPLETAR: Cambiar a uint16_t y corregir
    return 0;
}
```

 **Compilador:** onlinegdb.com, codechef.com, godbolt.org

 **Tiempo:** 10 minutos

Toma de Decisiones con `if`, `else if`, `else`

Permite que el flujo del diagrama se bifurque por dos ramas diferentes en el punto de la toma de decisión.

Estructura Básica

```
if (condicion1) {  
    // Código si condicion1 es verdadera  
} else if (condicion2) {  
    // Código si condicion2 es verdadera  
} else {  
    // Código si ninguna es verdadera  
}
```

Ejemplo en Embebido

```
uint16_t temperatura = leer_sensor();  
  
if (temperatura > TEMP_MAX) {  
    apagar_calefactor();  
    activar_alarma();  
} else if (temperatura < TEMP_MIN) {  
    encender_calefactor();  
} else {  
    // Todo normal  
}
```

 **UL 60730:** Siempre incluir `else` para manejar casos inesperados

⚡ Evaluación de Cortocircuito

✅ ¿Cómo funciona?

C evalúa de **izquierda a derecha** y **detiene** la evaluación cuando el resultado ya está definido.

```
// AND (&&): Si la primera es falsa, no evalúa la segunda
if (ptr != NULL && ptr->dato == 5) {
    // Seguro: ptr no es NULL antes de acceder
}

// OR (||): Si la primera es verdadera, no evalúa la segunda
if (error_critico || leer_sensor() > MAX) {
    // leer_sensor() solo se ejecuta si error_critico es falso
}
```

⚠ Impacto en Hardware

- Leer un registro de hardware puede tener **efectos secundarios**
- Si se usa en una condición con cortocircuito, **puede no leerse**
- ¡Esto puede causar fallos intermitentes!

```
// ❌ MAL: Puede que no se lea el registro
if (flag && (*REGISTRO & BIT_ALARMA)) {
    manejar_alarma();
}

// ✅ BIEN: Leer primero, luego evaluar
uint8_t estado_reg = *REGISTRO;
if (flag && (estado_reg & BIT_ALARMA)) {
    manejar_alarma();
}
```

🤖 Operadores Lógicos vs Operadores de Bits

📌 Operadores Lógicos

- `&&` AND lógico
- `||` OR lógico
- `!` NOT lógico

Resultado: **true (1) o false (0)**

```
if (temperatura > 50 && presion < 10) {  
    alarma();  
}
```

📌 Operadores de Bits

- `&` AND bit a bit
- `|` OR bit a bit
- `~` NOT bit a bit
- `^` XOR bit a bit

Resultado: **valor numérico**

```
uint8_t estado = *REGISTRO;  
if (estado & BIT_ALARMA) {  
    alarma();  
}
```

💡 **Regla:** Usar lógicos para condiciones, bits para máscaras de registros

RETO: Validación de Enclavamiento (Interlock)

 **OBJETIVO:** Verificar tres condiciones de seguridad usando lógicos y bits

ENUNCIADO:

"Escribe un condicional `if` que verifique tres condiciones de seguridad simultáneas usando operadores lógicos. Luego, usa operadores de bits para leer un estado específico dentro de un registro simulado."

```
//  CÓDIGO BASE
#include <stdint.h>
#include <stdbool.h>
#include <stdio.h>

// Simulación de registro de estado
uint8_t registro_estado = 0b10100101;

// COMPLETAR: Definir máscaras para:
// - BIT_PUERTA_CERRADA (bit 0)
// - BIT_NIVEL_AGUA_OK (bit 3)
// - BIT_TEMPERATURA_OK (bit 5)

// COMPLETAR: Escribir un if que verifique las 3 condiciones
// usando && y también usando máscaras de bits
```

 **Compilador:** onlinegdb.com, codechef.com, godbolt.org

 **Tiempo:** 10 minutos

Estructuras de Repetición

Todo ciclo debe terminar luego de repetirse un número finito de veces. Dentro del conjunto de instrucciones siempre debe existir una condición de parada o fin de ciclo. En cada iteración del mismo son evaluadas las condiciones necesarias para decidir si se debe seguir ejecutando o si debe detenerse.

for

Iteraciones **contadas**

```
for (int i = 0; i < 10; i++) {  
    procesar(i);  
}
```

✓ Útil para arrays, tiempos fijos

while

Basado en **condición**

```
while (bandera == 0) {  
    esperar_interrupcion();  
}
```

✓ Ideal para hardware, sensores

do-while

Ejecuta **al menos una vez**

```
do {  
    leer_sensor();  
} while (dato_no_valido);
```

✓ Lectura inicial obligatoria

⚠ **UL 60730:** Evitar bucles infinitos sin watchdog

switch-case - Selección Múltiple

Permite que el flujo del diagrama se bifurque por varias ramas en el punto de la toma de decisión. La elección del camino a seguir depende del contenido de la variable conocida como selector, la cual puede tomar valores de un conjunto previamente establecido.

✓ Estructura Limpia

```
switch (estado_motor) {  
    case ESTADO_APAGADO:  
        apagar_motor();  
        break;  
    case ESTADO_ENCENDIDO:  
        encender_motor();  
        break;  
    case ESTADO_FALLA:  
        manejar_falla();  
        break;  
    default:  
        estado_indeterminado();  
        break;  
}
```

⚠ Reglas UL 60730

- **SIEMPRE** usar `break`
- **SIEMPRE** incluir `default`
- `default` debe llevar al sistema a un **estado seguro**

```
default:  
    apagar_sistema();  
    activar_alarma();  
    break;
```

RETO: Main Loop y Máquina de Estados

 **OBJETIVO:** Implementar un bucle principal con switch-case para control de estados

ENUNCIADO:

"Implementa un bucle principal `while(1)` simulado. Dentro de él, coloca un `switch-case` que controle los estados de un aparato (REPOSO, TRABAJO, FALLA). Usa un contador con un ciclo `for` para simular el paso del tiempo y cambiar de estado."

```
// 🖥️ CÓDIGO BASE
#include <stdint.h>
#include <stdio.h>

// COMPLETAR: Definir enum con estados
// COMPLETAR: Variable de estado
// COMPLETAR: Bucle while(1) con switch
// COMPLETAR: for para simular tiempo

int main() {
    // Inicializar estado
    while(1) {
        // switch-case para manejar estados
    }
    return 0;
}
```

 **Compilador:** onlinegdb.com, codechef.com, godbolt.org

 **Tiempo:** 12 minutos

Funciones

En el lenguaje de programación C la solución de un problema se expresa por medio de un programa; la solución de un subproblema, por medio de una **función**. El uso de funciones tiene múltiples ventajas: facilitan la lectura y escritura de los programas, permiten el trabajo en paralelo, facilitan la asignación de responsabilidades, facilitan el mantenimiento.

Estructura

```
// Prototipo (declaración)
uint16_t calcular_promedio(uint16_t a, uint16_t b);

// Definición (implementación)
uint16_t calcular_promedio(uint16_t a, uint16_t b) {
    return (a + b) / 2;
}
```

- **Tipo de retorno:** uint16_t
- **Argumentos:** (uint16_t a, uint16_t b)
- **Cuerpo:** Código entre { }

Paso por Valor

Los parámetros por valor permiten pasar datos entre el programa principal y las funciones, e incluso entre las mismas funciones.

En el parámetro se escribe una copia de la variable original.

Si el parámetro sufre una alteración en la función que lo recibe, la variable original no se ve afectada.

El argumento se **copia** en el stack. La función trabaja con una **copia** local.

```
void incrementar(uint8_t valor) {  
    valor++; // Solo modifica la copia local  
}  
  
uint8_t x = 5;  
incrementar(x);  
// x sigue siendo 5
```

💡 **Ventaja:** No modifica la variable original

💡 **Desventaja:** Copia datos (puede ser costoso para estructuras grandes)

RETO: Configurador de Periféricos

 **OBJETIVO:** Crear función que use enum para configurar velocidad de motor

ENUNCIADO:

"Crear una función encargada de cambiar la velocidad de un motor. La función debe recibir como argumento una enumeración (enum) con las velocidades permitidas (BAJA, MEDIA, ALTA) y retornar un código de éxito o error si el parámetro es inválido."

```
//  CÓDIGO BASE
#include <stdint.h>
#include <stdbool.h>
#include <stdio.h>

// COMPLETAR: Definir enum VelocidadMotor_t
// COMPLETAR: Función motor_set_velocidad()
// COMPLETAR: main() que pruebe la función
```

 **Compilador:** onlinegdb.com, codechef.com, godbolt.org

 **Tiempo:** 10 minutos

Paso por Referencia

En lugar de escribir una copia de la variable en el parámetro se escribe la dirección de la misma. Si el parámetro sufre una alteración en la función que lo recibe, la variable original también se ve afectada. En C, las llamadas por referencia se realizan mediante **punteros**.

✓ Modificar variables externas

```
void incrementar(uint32_t* contador) {  
    (*contador)++; // Modifica la variable original  
}  
  
uint32_t ciclos = 1000;  
incrementar(&ciclos);  
// ciclos ahora es 1001
```

✓ Ventaja: No copia datos, modifica directamente

⚠ Riesgos y Mitigaciones

- **Puntero NULL:** Acceso a dirección 0 → Hard Fault
- **Siempre validar:**

```
void incrementar_seguro(uint32_t* contador) {  
    if (contador == NULL) {  
        return; // No hacer nada  
    }  
    (*contador)++;  
}
```

⚠ UL 60730: Validar punteros antes de usar

📌 Punteros - Direcciones de Memoria

Es una variable que contiene la dirección de memoria de otra variable y se representa por medio de operadores de **dirección (&)** e **indirección (*)**.

📌 Conceptos Básicos

- **&** → Operador de dirección
- ***** → Operador de desreferencia o indirección
- **Tipo** → `uint8_t*` puntero a `uint8_t`

```
uint8_t variable = 42;
uint8_t* puntero = &variable; // Guarda dirección

uint8_t valor = *puntero;      // Lee valor = 42
*puntero = 100;                // Modifica variable
```

💡 Visualización

```
variable → 0x1000 [ 42 ]
puntero → 0x2000 [ 0x1000 ]
*puntero → accede a 0x1000 → obtiene 42
```

RETO: Modificador de Contador

 **OBJETIVO:** Usar punteros para modificar una variable externa

ENUNCIADO:

"Escribir una función que reciba la dirección de memoria de una variable que representa el contador de ciclos de un electrodoméstico. La función debe incrementar el valor directamente en la memoria externa usando desreferencia."

```
// 🖥️ CÓDIGO BASE
#include <stdint.h>
#include <stdio.h>

// COMPLETAR: Función que recibe puntero a uint32_t
// y lo incrementa en 1

int main() {
    uint32_t contador_ciclos = 0;
    // COMPLETAR: Llamar a la función
    // COMPLETAR: Mostrar resultado
    return 0;
}
```

 **Compilador:** onlinegdb.com, codechef.com, godbolt.org

 **Tiempo:** 10 minutos



static - Control de Visibilidad y Persistencia

Son variables similares a las locales, pero conservan su valor durante la ejecución del programa. Es decir, comienzan a existir cuando se llama a la función y conservan su valor aun cuando el control regresa al programa principal.

✓ static en variables locales

Preserva el valor entre llamadas a la función

```
void contar_llamadas() {  
    static uint32_t contador = 0;  
    contador++;  
    printf("Llamada #%lu\n", contador);  
}
```

✓ static en globales/funciones

Oculto la variable/función fuera del archivo .c

```
// modulo.c  
static uint8_t estado_interno = 0;  
  
static void funcion_privada() {  
    // Solo visible aquí  
}  
  
// ✓ Función pública  
void modulo_iniciar() {  
    funcion_privada();  
}
```

💡 Encapsulamiento al estilo C

⚡ **volatile** - No Optimizar

Le indica al compilador que el valor de una variable puede cambiar de forma inesperada. Esto evita que el compilador haga "optimizaciones" y lo obliga a leer siempre el valor directamente desde la memoria RAM, en lugar de usar una copia temporal guardada en un registro del procesador.

⚠ **Problema: Optimización del Compilador**

```
uint8_t bandera = 0;

// Bucle de espera
while (bandera == 0) {
    // El compilador puede optimizar esto a:
    // while(1) { } → ¡Nunca lee la RAM!
}
```

✗ La variable puede cambiar por una ISR, pero el compilador **no lo sabe**

✅ **Solución: volatile**

```
volatile uint8_t bandera = 0;

while (bandera == 0) {
    // El compilador SIEMPRE lee de RAM
}
```

✅ Obliga al microcontrolador a leer la RAM en **cada iteración**

📌 **Casos Obligatorios**

- Variables modificadas por **ISR**
- Variables compartidas entre **hilos/tareas**
- Registros de **hardware** mapeados en memoria

RETO: Simulación de Interrupción

 **OBJETIVO:** Demostrar la diferencia entre con y sin volatile

ENUNCIADO:

"Escribir un bucle `while` que dependa de una bandera global. Demostrar cómo el compilador optimiza el bucle si la variable no es `volatile`, y cómo cambia el código al agregar el modificador `volatile` obligando al microcontrolador a leer la RAM en cada iteración."

```
// 📄 CÓDIGO BASE
#include <stdint.h>
#include <stdio.h>

// COMPLETAR: Variable bandera con y sin volatile
// COMPLETAR: Función que simula ISR
// COMPLETAR: Bucle while que espera la bandera

int main() {
    // Simular interrupción después de 5 iteraciones
    return 0;
}
```

 **Compilador:** onlinegdb.com, codechef.com, godbolt.org

 **Tiempo:** 12 minutos



struct - Agrupar Datos

Es una colección de diferentes variables agrupadas bajo un mismo nombre. A diferencia de un arreglo (array), que solo guarda datos del mismo tipo, una estructura te permite combinar tipos de datos distintos (como int, float y char) en un solo bloque de memoria.

Definición

```
typedef struct {  
    uint16_t temperatura; // 2 bytes  
    uint16_t humedad;     // 2 bytes  
    uint32_t timestamp;   // 4 bytes  
    uint8_t  estado;      // 1 byte  
} SensorData_t;
```

Uso en Embebido

```
SensorData_t sensor;  
sensor.temperatura = 25;  
sensor.humedad = 60;  
sensor.timestamp = get_time();  
sensor.estado = 1;  
  
// Pasar a función  
procesar_sensor(&sensor);
```

✅ Representa entidades complejas (sensor, motor, compresor)

union - Compartir Memoria

Es muy similar a una `struct`, pero con una diferencia fundamental: todos sus miembros comparten la misma posición de memoria.

Mientras que una estructura reserva espacio para cada una de sus variables, una unión solo reserva espacio para el miembro más grande. Esto significa que solo puedes guardar el valor de un miembro a la vez. Si guardas un dato en un miembro, se borrará el contenido del miembro anterior.

Definición

```
typedef union {
    uint32_t valor_completo;
    uint8_t bytes[4];
} DatoUnion_t;
```

✅ Todos los miembros comparten la **misma dirección de memoria**

Mapeo de Protocolos

```
DatoUnion_t trama;
trama.valor_completo = 0x12345678;

// Transmitir por UART
for (int i = 0; i < 4; i++) {
    uart_transmitir(trama.bytes[i]);
}
// Envía: 0x12, 0x34, 0x56, 0x78
```

✅ Útil para tramas de comunicación (UART, SPI, I2C)

⚠️ Depende del endianness del micro

sizeof - Tamaño en Tiempo de Compilación

Es un operador unario en C que se utiliza para calcular el **tamaño exacto en bytes** que ocupa una variable o un tipo de dato en la memoria del sistema.

A diferencia de la mayoría de las funciones en C, `sizeof` no se procesa cuando el programa se está ejecutando, sino que el compilador lo calcula **durante la compilación** y lo reemplaza por un número fijo.

Uso

```
size_t tam = sizeof(SensorData_t);
printf("Tamaño: %lu bytes\n", tam);

// Para arrays
uint8_t buffer[100];
size_t len = sizeof(buffer); // 100
```

Importante

- Se evalúa en **tiempo de compilación** (no runtime)
- Esencial para **calcular padding**
- Útil para **copias de memoria** (`memcpy`)

```
// Copia segura de estructura
SensorData_t origen, destino;
memcpy(&destino, &origen, sizeof(SensorData_t));
```

RETO: Mapeador de Protocolos

 **OBJETIVO:** Usar union para mapear struct a bytes

ENUNCIADO:

"Crear una `union` que contenga una `struct` con los parámetros de configuración de un electrodoméstico y, al mismo tiempo, un arreglo de bytes (`uint8_t buffer[]`). Demostrar cómo modificar un dato de la estructura altera automáticamente los bytes del arreglo."

```
// 🖥️ CÓDIGO BASE
#include <stdint.h>
#include <stdio.h>

// COMPLETAR: Definir struct Config_t
// COMPLETAR: union con struct y buffer de bytes
// COMPLETAR: main() que modifique la struct y muestre bytes
```

 **Compilador:** onlinegdb.com, codechef.com, godbolt.org

 **Tiempo:** 15 minutos

Padding en Micros de 32 bits

El padding (relleno de memoria) es un concepto de bajo nivel que consiste en insertar **bytes vacíos o "basura"** entre las variables de una estructura, para alinearlas con los límites naturales de acceso de la arquitectura del procesador.

Es una acción **automática** que realiza el **compilador** antes de generar el programa final.

Estructura Desordenada

```
struct Desordenada {  
    uint8_t a;    // offset 0  
    uint32_t b;   // offset 4 (padding 1-3)  
    uint16_t c;   // offset 8  
    uint8_t d;    // offset 10  
}; // sizeof = 12 bytes
```

 Desperdicio: 33% de RAM

Estructura Optimizada

```
struct Optimizada {  
    uint32_t b;   // offset 0  
    uint16_t c;   // offset 4  
    uint8_t a;    // offset 6  
    uint8_t d;    // offset 7  
}; // sizeof = 8 bytes
```

 Ahorro: 4 bytes (33% menos)

 **Regla:** Ordenar de **mayor a menor** tamaño

Macros con Argumentos

Es una directiva del preprocesador en C que funciona como una "plantilla" de texto. Te permite definir un bloque de código con parámetros que se reemplaza directamente en tu archivo fuente **antes de que el código sea compilado**.

A diferencia de una función normal, una macro no genera un salto en la memoria ni reserva espacio en el stack. Simplemente hace un **copiar y pegar inteligente** de texto.

⚠ Peligros

```
#define CUADRADO(x) x*x

// ❌ MAL: Se expande como 3+2*3+2 = 11
int resultado = CUADRADO(3+2);
```

❌ Efectos secundarios sintácticos

✅ Macros Seguras

```
#define CUADRADO_SEGURO(x) ((x)*(x))

// ✅ BIEN: ((3+2)*(3+2)) = 25
int resultado = CUADRADO_SEGURO(3+2);

// ✅ Siempre usar paréntesis
#define MAX(a,b) ((a) > (b) ? (a) : (b))
```

✅ Paréntesis **protectores** en todos los argumentos

⚡ Funciones `inline`

Es una función normal en C que lleva la palabra clave `inline` al principio de su definición. Le sugiere al compilador que, en lugar de generar una llamada tradicional a la función, **copie y pegue el código completo de la función** directamente en el lugar donde se la invoca.

Nació como una evolución segura para reemplazar a las "macros con argumentos", buscando la velocidad de una macro pero con la seguridad de una función real.

📌 Qué es

Solicitud al compilador para **sustituir el código** en lugar de hacer una llamada a función.

```
static inline uint16_t leer_temp() {  
    return *TEMP_REG;  
}
```

⚖ Balance

Ventaja

✓ Más rápido

✓ Sin overhead de llamada

✓ Bueno para funciones pequeñas

Desventaja

✗ Más código en Flash

✗ Puede inflar el binario

✗ Mala para funciones grandes

💡 Usar para **getters/setters** y funciones **críticas por tiempo**

RETO: Optimización de Espacio

🎯 **OBJETIVO:** Reordenar estructura para eliminar padding

📝 ENUNCIADO:

"Diseñar una estructura de datos mal alineada que mezcle tipos de 8, 16 y 32 bits. Comprobar su tamaño inflado con `sizeof`. El reto consiste en reordenar los miembros para reducir el tamaño de la estructura al mínimo posible, eliminando los bytes de padding."

```
// 📄 CÓDIGO BASE
#include <stdint.h>
#include <stdio.h>

// Estructura desordenada (actual)
struct Sistema {
    uint8_t estado;      // 1 byte
    uint32_t contador;   // 4 bytes
    uint16_t temperatura; // 2 bytes
    uint8_t error;       // 1 byte
}; // Tamaño actual: ?

// COMPLETAR: Reordenar para minimizar tamaño
// COMPLETAR: Mostrar tamaño antes y después
```

🔗 **Compilador:** onlinegdb.com, codechef.com, godbolt.org

🕒 **Tiempo:** 10 minutos

¡Felicidades! Curso Completado

✓ Lo que Aprendiste

- ✓ Palabras reservadas
- ✓ Tipos de datos portables (`stdint.h`)
- ✓ Operadores lógicos vs bits
- ✓ Máquinas de estado con `switch-case`
- ✓ Funciones y `enum` para parámetros legibles
- ✓ Punteros y paso por referencia
- ✓ `static` y `volatile` para firmware robusto
- ✓ `struct` y `union` para datos y protocolos
- ✓ Optimización de padding y macros seguras

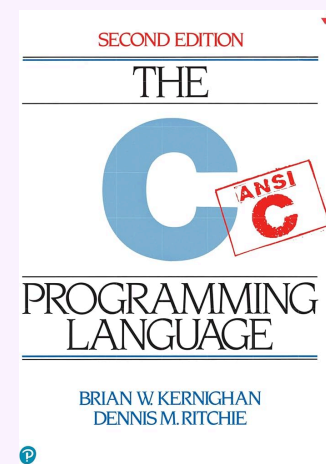
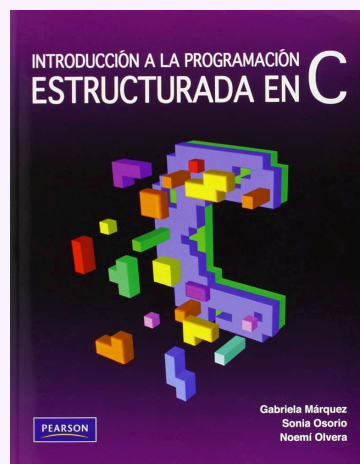
Recursos Adicionales

- **Documentación:** UL 60730 Standard
- **Documentación:** MISRA Standard
- Libro: ["Making Embedded Systems" - Elecia White](#)
- **Herramientas:** Compilador Online, IDE, Debugger

Próximos Pasos

- Implementar en hardware real (STM32, PIC, etc.)
- Agregar Watchdog Timer
- Implementar comunicación UART para diagnóstico

Bibliografía Recomendada



Preguntas y Respuestas - Q&A

RECAPITULANDO

 Tipos de Datos Operadores Bucles y Switch Funciones Punteros static/volatile struct/union Padding/Macrosgithub.com/AngelTepetla/Curso-Basico-Cjangel.tepetla@gmail.com

¡GRACIAS POR SU PARTICIPACIÓN!

Curso de Firmware - C para Sistemas Embebidos de 32 bits

[Volver al inicio](#)

J. Tepetla · 2026

Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0)